

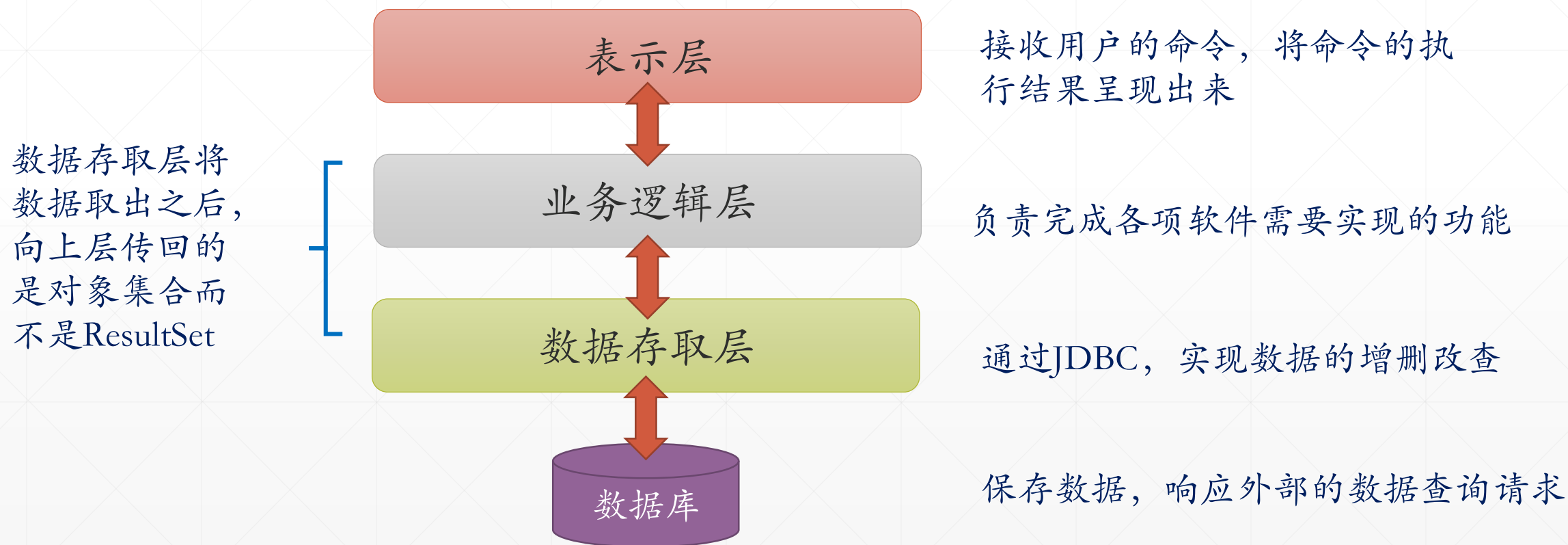


数据库应用程序编程套路

北京理工大学计算机学院
金旭亮

分层的数据库应用程序架构

在实际开发中，为了提升代码的可维护性，人们通常将程序分层。



处理对象，而不是数据集



在基于数据库的应用中，我们通常会把从数据库中提取出来的数据转换为对象，应用程序业务逻辑层以上的各层组件，处理的是对象，而不是数据集。



在JDBC中，实际上就是把结果集转换为JavaBean对象集合。

JavaBean示例



其基本格式为：私有字段 + getter/setter 方法



如果需要支持序列化，则JavaBean还应该实现Serializable接口

```
public class Admin {  
    private int adminId;  
  
    public int getAdminId() {  
        return adminId;  
    }  
    public void setAdminId(int adminId) {  
        this.adminId = adminId;  
    }  
}
```

IntelliJ IDEA等IDE都支持提取类的字段信息，自动地生成getter/setter方法，还有一个Lombok开源工具，能以注解的方式生成getter/setter方法。

使用JavaBean封装数据

//数据实体类

```
public class OrderClient {  
    private int clientID = 0;  
    private String clientName = "";  
    private String address = "";
```

getter和setter方法

@Override

```
public String toString() {...}
```

```
}
```

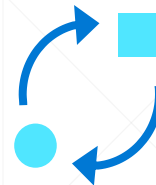


Output OrderClient ×			
24 rows			
	ClientID	ClientName	Address
1	2	李四	上海
2	3	王五	重庆
3	4	赵六	广州

表变成数据实体类（JavaBean），表中的字段变成类的属性。

表中的每一行，对应着一个JavaBean对象。

怎样将结果集转换为JavaBean集合?



ResultSet



List<JavaBean>

非常简单，先创建好一个空的JavaBean对象集合，然后写一个循环结构，遍历结果集中的每一条记录，针对每一条记录，只需new一个JavaBean，然后从结果集中按字段提取数据，填充到JavaBean的相应字段中，将填充好的JavaBean追加到对象集合中，最后再return给外界即可。

从ResultSet到对象集合

// 查询表中保存的数据

```
ResultSet rs = statement.executeQuery();  
while (rs.next()) {  
    clients.add(OrderClientRepositoryHelper.getOrderClientFromResultSet(rs));  
}
```

OrderClientRepository.java



//从ResultSet当前记录中读取数据, 创建一个OrderClient对象, 返回给外界

```
public static OrderClient getOrderClientFromResultSet(ResultSet rs)  
    throws SQLException {  
    if (rs == null) {  
        return null;  
    }  
    OrderClient client = new OrderClient();  
    client.setClientID(rs.getInt(columnLabel: "ClientID"));  
    client.setClientName(rs.getString(columnLabel: "ClientName"));  
    client.setAddress(rs.getString(columnLabel: "Address"));  
    return client;  
}
```

OrderClientRepositoryHelper.java

Repository设计模式



Repository设计模式是一种常用于数据存取层的设计模式，应用很广.....



1

将要实现的CRUD功能抽象为方法，集中放到一个接口中。

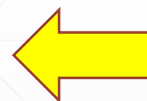
2

Repository实现这个接口，在内部封装特定的数据存取技术，实现接口所定义的各种方法。

使用Repository模式封装数据存取功能

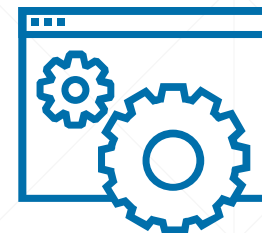


```
//定义与客户相关的CRUD功能
public interface IOrderClientRepository {
    //按照客户姓名进行模糊检索, 返回符合条件的OrderClient集合
    List<OrderClient> getClients(String firstName);
    //添加一条客户记录
    int addClient(OrderClient client);
    //按照id查找指定的客户记录
    OrderClient getClient(int id) throws SQLException;
    //更新指定的客户记录
    int updateClient(OrderClient client);
    //删除指定id的客户记录
    int deleteClient(int clientID);
}
```



将需要的CRUD功能抽象为一个接口，注意输入输出除了少量原始数据类型，基本上都是JavaBean对象或对象集合。

实现Repository



```
public class OrderClientRepository
    implements IOrderClientRepository, Closeable {

    private Connection connection = null;
    private final String connectionStr = "jdbc:sqlite:test.db";

    public OrderClientRepository() throws SQLException {...}

    @Override
    public List<OrderClient> getClients(String firstName) {...}
    @Override
    public int addClient(OrderClient client) {...}
    @Override
    public int updateClient(OrderClient client) {...}
    @Override
    public int deleteClient(int clientID) {...}
    public OrderClient getClient(int id) throws SQLException {...}
    @Override
    public void close() throws IOException {...}

}
```

Repository实现类不仅要实现对应的Repository接口，通常还需要实现Closeable接口，以便自动释放相应的资源

在本例中，封装的是JDBC，访问的是SQLite数据库，在实际开发中，完全可以封装其他的数据存取技术，访问其他类型的数据库，甚至是从网络上提取数据.....

典型的CRUD功能代码

```
@Override
public List<OrderClient> getClients(String firstName) {
    List<OrderClient> clients = new ArrayList<>();
    String sql = "select * from OrderClient where ClientName Like ? ";
    try(PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setString(1, "%" + firstName + "%");
        // 查询表中保存的数据
        ResultSet rs = statement.executeQuery();
        // 遍历结果集, 填充OrderClient集合
        while (rs.next()) {
            clients.add(OrderClientRepositoryHelper.getOrderClientFromResultSet(rs));
        }
    } catch (SQLException e) {
        System.err.println(e.getMessage());
    }
    return clients;
}
```

此方法在内部封装JDBC代码从数据库中提取数据，然后向外界返回一个OrderClient集合。

使用Repository查询数据的示例代码



```
public class OrderClientRepositoryTest {  
  
    public static void main(String[] args) throws SQLException {  
        IOrderClientRepository repository = new OrderClientRepository();  
        //查找姓“李”的客户  
        List<OrderClient> clients = repository.getClients( firstName: "李");  
        //输出查找到的结果  
        for (OrderClient orderClient : clients) {  
            OrderClientRepositoryHelper.printOrderClient(orderClient);  
        }  
    }  
}
```



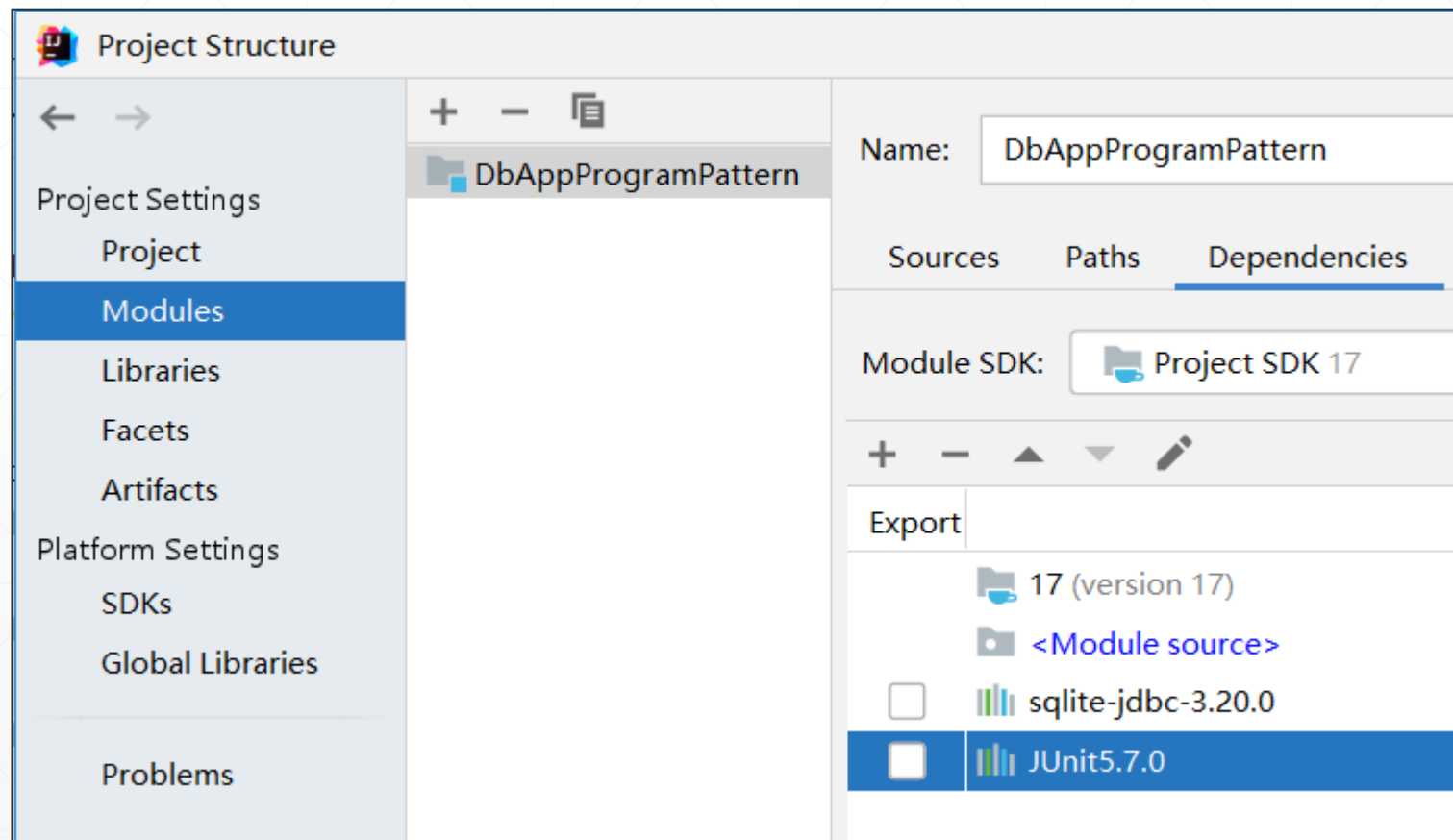
由于有一个Repository，外界要提取数据时，只需创建一个它的实例，调用它相应的方法，就可以获取到相应的实体对象（或实体集合），根本不需要知道这些数据从哪来，使用的是什麼数据存取技术，从而很好地实现了“向上层组件屏蔽底层技术细节”这一系统设计目标。

测试Repository功能的正确性

在实际开发中，通常会使用单元测试框架对Repository中的功能代码进行测试。在Java应用程序中，最常用的单元测试框架就是JUnit。

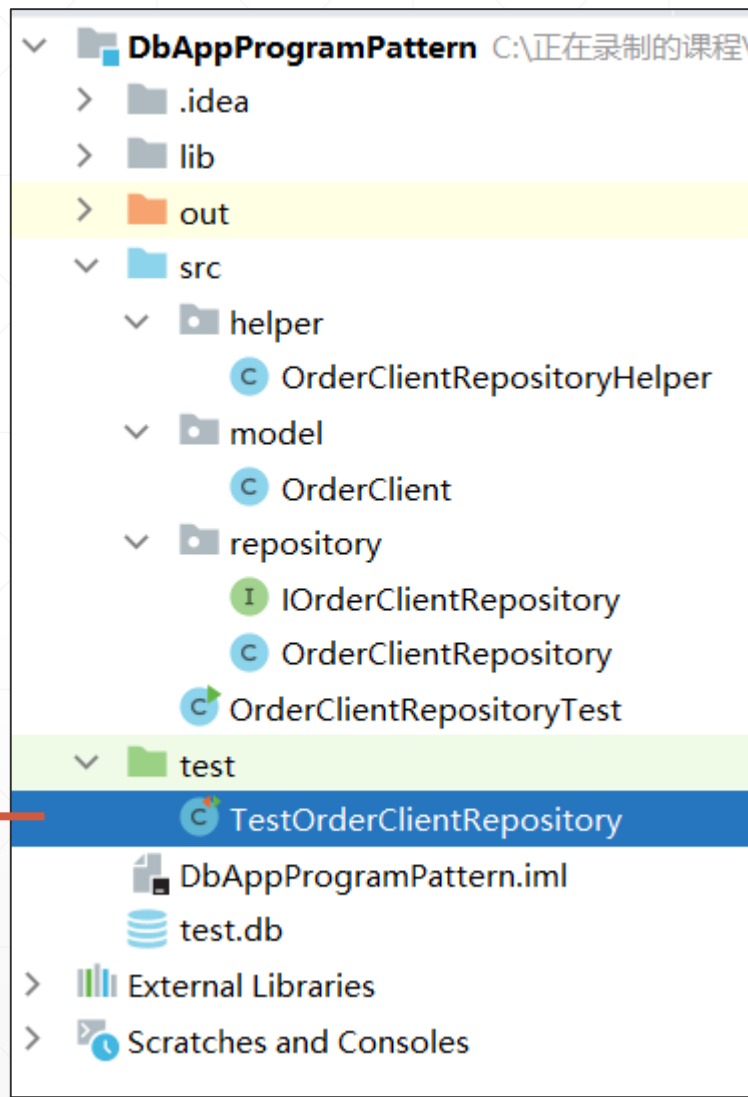


示例项目中导入了JUnit5对Repository进行测试。



典型的单元测试代码编写套路

```
public class TestOrderClientRepository {  
  
    private OrderClientRepository repo = null;  
    private Random ran = new Random();  
  
    //每个测试前, 创建Repository实例  
    @BeforeEach  
    public void setUp() throws Exception {  
        repo = new OrderClientRepository();  
        ran = new Random();  
    }  
  
    //测试结束后, 关闭Repository  
    @AfterEach  
    public void tearDown() throws Exception {  
        repo.close();  
    }  
}
```



编写和运行单元测试



```
@Test
public void testGetClients() {
    //调用要被测试的代码
    List<OrderClient> clients = repo.getClients( firstName: "");
    for (OrderClient orderClient : clients) {
        System.out.println(orderClient);
    }
    //检查结果是否正确
    assertTrue( condition: clients.size() > 0);
}
```

Run: TestOrderClientRepository.testGetClients x

Tests passed: 1 of 1 test – 176 ms

Test Results	176 ms	"C:\Program Files\Java\jdk-17\bin\java.exe" ...
TestOrderClientRepository	176 ms	OrderClient [clientId=2, clientName=李四, address=上海]
testGetClients()	176 ms	OrderClient [clientId=3, clientName=王五, address=重庆]
		OrderClient [clientId=4, clientName=赵六, address=广州]

小结



本讲所介绍的Repository是实际开发中构建数据存取层的常用模式，一定要注意掌握。



有关JDBC的内容还有很多，但由于我们在实际开发中现在大多是基于各种框架进行开发，直接使用JDBC的并不算多，因此，我们只需要了解其基本原理就够了，感兴趣的同学可以课后通过钻研相关书籍，了解与掌握JDBC的更多知识。



学完了JDBC，再与JavaFX等Java平台的GUI框架结合起来，你就具备了开发一个功能完备的数据库应用程序的基础。现在请动手，打造一个属于你的实用的伟大的数据库应用程序吧！

作业



1

本讲示例只实现了对示例数据库的查询功能，请在本讲示例的基础之上，完善Repository的功能，实现CURD四种操作。

2

本讲示例是针对SQLite数据库实现的，请将本讲示例进行移植，使用一个你所熟悉其他关系类型数据库存储数据。

作业



使用学过的知识，结合JavaFX和Swing等技术，编写一个Java桌面应用程序，它可以连接一个关系型数据库，显示它所包容的所有表，并且可以浏览特定表所包容的数据。

下一讲



Spring JDBC精要